# TraceGraph-Bench: Auditing LLM Agents with Graph-Level Anomaly Detection over Tool-Call Trajectories

Anonymous Author(s)

## ABSTRACT

LLM agents increasingly execute multi-step tool workflows on the Web—browsing, writing, paying, and posting on a user's behalf—and every step they take is mediated by tool calls whose arguments and results flow across an untrusted observation channel. Existing safety auditing operates at the wrong granularity: token-level filters and step-level guardrails inspect individual outputs, yet the compromises that matter on the agentic Web, from indirect-prompt-injection hijacks to over-privileged tool use, unfold as *structural* patterns across an entire trajectory. We introduce TraceGraph-Bench, the first benchmark for *graph-level* anomaly detection over agent trajectories: each trace is compiled into a heterogeneous provenance graph whose nodes are tasks, model steps, tool calls, argument entities, results, and sensitive sinks, and whose edges carry invocation, argument-flow, and temporal structure. The benchmark couples three anomaly families—injection-hijacked trajectories inherited from AgentDojo and AgentDrift, malicious-use trajectories from AgentHarm, and a newly constructed over-privileged tool-use family—with an evaluation protocol that demands leave-one-attack-family-out generalization, cross-environment transfer, mimicry robustness, and an explicit disclosure ablation between structure-only and structure-plus-text inputs. Across detector families spanning hand-crafted rules, sequence models, node-level graph anomaly detection, and graph-level detection, the strongest structure-only detector attains [TBD] AUPRC on seen families but drops to [TBD] under LOAO, and all detectors degrade by at least [TBD] under mimicry, exposing a structural blind spot that no text-level detector closes. We release traces, graphs, and harness.

## KEYWORDS

graph anomaly detection, LLM agents, provenance graphs, benchmark, Web safety auditing

**ACM Reference Format:**

Anonymous Author(s). 2026. TraceGraph-Bench: Auditing LLM Agents with Graph-Level Anomaly Detection over Tool-Call Trajectories. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 INTRODUCTION

Large language model (LLM) agents have crossed from answering questions to acting on the Web: they browse pages, edit documents, move money, and post content by orchestrating sequences of tool calls against external services [4, 9]. This deployment shift changes what safety means. In a chatbot, a failure is a harmful utterance; in an agent, a failure is a *deed*—a privileged tool invoked at the wrong time, with the wrong arguments, for the wrong reason—and deeds are composed of many individually innocuous steps. Recent multi-turn audits show that this gap between what agents can do and what they should do widens precisely as interactions lengthen and tool access deepens [19].

The threat surface of the agentic Web has likewise matured into families rather than incidents. Indirect prompt injection turns any tool result, webpage, or document the agent reads into an attack channel, with automatically generated and covert payloads demonstrated at scale [18, 23, 41, 42]; adversarial prompts can even steer which tools the agent selects [36]. Separately, malicious users can request harmful goals outright, and benchmark suites now measure agent harmfulness and safety awareness across tool stages [3, 48, 49]. A third family receives less attention but is arguably more damaging in deployment: over-privileged tool use, where an agent performs an action that the task did not warrant, without any injected payload at all [46]. Defenses have proliferated—structured queries [6], preference-optimized resistance [7], tool-dependency-graph plan conformance [2], and step-level guardrails and diagnostic frameworks [5, 15, 20, 30]—yet almost all of them judge the agent one response or one step at a time.

We argue that this per-step granularity is the wrong audit unit, and that the systems-security community settled the analogous question a decade ago. Host intrusion detection moved from signature matching on individual syscalls to *provenance graphs*, because attacks such as data exfiltration are invisible in any single event and emerge only from the connectivity of many events [53]. The same structural insight transfers to agents: an injection hijack is a dataflow edge from an untrusted observation into a sensitive sink; over-privilege is a privileged call whose authorizing subgraph is missing; a malicious agent is a workflow whose entire call structure diverges from task service. Text-level auditors cannot see any of this, because the pattern is not in the tokens—it is in the graph. Provenance-graph detectors, however, also teach a cautionary lesson: mimicry attacks that splice benign structure into attack structure defeat even graph-based detectors [12], so any credible audit protocol must test robustness, not just accuracy [13].

Translating this insight into a benchmark surfaces three obstacles that, to our knowledge, no prior work has addressed together. First, agent traces are *sequences with semantics*, and the closest existing resources model them as such: AgentDrift publishes step-labeled injection-hijacked trajectories and DriftNet detects and localizes hijacks with a trajectory transformer [31, 32], while benchmark suites such as AgentSecBench, HarnessRisk, PASTABench, AutoDojo, and WAInjectBench evaluate agents or detectors on textual trajectories [1, 4, 24, 28, 37]. None of them constructs the provenance graph

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

*Conference'17, July 2017, Washington, DC, USA*

© 2026 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

that makes structural auditing possible. Second, graph anomaly detection (GAD) has matured from node-level benchmarks [22, 38] to graph-level and foundation-model methods [34, 43], but its standard testbeds—citation and transaction networks—share none of the temporal, typed, small-graph structure of agent traces, so headroom on those testbeds says little about the agentic Web. Third, evaluation practice itself is under audit: prompt-injection detectors have been shown to pass the very tests they were tuned on [25], and agent-safety benchmarks have been criticized for weak construct validity [44]; a new benchmark must therefore build generalization and contamination controls into its spine rather than appending them as afterthoughts.

This paper introduces TraceGraph-Bench, a benchmark and evaluation protocol that compiles agent tool-call trajectories into heterogeneous provenance graphs and treats trajectory-level auditing as graph-level anomaly detection. Our data engine couples three anomaly families under one graph schema: injection-hijacked and benign trajectories replayed from AgentDojo and AgentDrift [9, 31], malicious-use trajectories from AgentHarm [3], and a newly constructed over-privileged tool-use family that operationalizes the taxonomy of [46]. A disclosure ablation contrasts structure-only graphs with structure-plus-text graphs, isolating what structure alone contributes. The protocol demands leave-one-attack-family-out (LOAO) generalization, cross-environment transfer between replayed and newly generated environments, robustness to mimicry-style splicing [12], and an audit-economics accounting of detector cost per trace. Our reference detector—a supervised heterogeneous-graph detector over the compiled graphs—together with a matrix of rule-based, sequence-based, node-level GAD, and graph-level GAD baselines yields a first systematized map of this new problem, showing [TBD—headline findings preview]. Our contributions are:

- **Trajectory provenance graphs.** A typed construction protocol that compiles raw agent traces into heterogeneous provenance graphs (task, model step, tool call, argument entity, result, sensitive sink; invocation, argument-flow, and temporal edges), with a disclosure knob that releases graphs at structure-only or structure-plus-text granularity.
- **The first graph-level auditing benchmark.** Three anomaly families under one schema, spanning injection hijack, malicious use, and over-privilege, with LOAO, cross-environment transfer, mimicry robustness, and audit-economics metrics designed against the contamination failure modes documented in prior audits.
- **A systematized detector matrix.** Baselines from hand-crafted rules and sequence models to repurposed node-level GAD and modern graph-level GAD, plus a supervised heterogeneous-graph reference detector, all evaluated on identical graph inputs.
- **Empirical findings on structural auditability.** Evidence on when structure alone suffices, when text is necessary, how detectors generalize to unseen attack families, and where mimicry breaks them [TBD—fill with concrete numbers].

Section 2 situates our contribution against agent safety, GAD, and provenance research; Sections 5 and 6 formalize the graph construction and benchmark protocol; Sections 7 and 8 report the study; Section 9 concludes.

## 2 RELATED WORK: AGENT SAFETY AND ATTACKS

*Indirect prompt injection on the agentic Web.* The defining attack channel of tool-using agents is the observation itself: instructions hidden in tool results, web pages, or documents steer the agent while the user sees nothing [41]. The line of work is advancing quickly. Automatic generators such as PIA produce task-agnostic injection suffixes that transfer across scenarios [23]; WebInject exploits the visual channel of web agents [42]; WAInjectBench systematically benchmarks injection detections for web agents and finds detectors tuned on one page layout fail on others [24]; and the newest work pushes toward covert payloads that keep the victim's visible behavior clean while exfiltrating data [18]. A parallel line attacks tool selection itself, causing the agent to invoke attacker-favored tools [36]. What these families share, and what motivates our benchmark, is that success is only fully visible in the resulting *action stream*: which tools were called, with which arguments, feeding which downstream calls.

*Malicious use and safety awareness.* A complementary threat model drops the adversary-in-the-middle and asks what happens when the user is the adversary. AgentHarm measures the harmfulness of agents that comply with malicious requests across 44 tool-enabled tasks and shows chat-level refusal training fails to transfer to agentic settings [3]; ToolSword traces safety issues across planning, execution, and reflection stages of tool learning [48]; R-Judge benchmarks whether models are even *aware* of safety risk in recorded multi-turn agent interactions [49]; and Unsafer extends the audit to multi-turn tool-using agents, showing risk compounds across turns and proposing step-level defenses [19]. AgentSecBench aggregates prompt injection, privacy leakage, and tool-use integrity into one measurable surface [1], and HarnessRisk evaluates the agent *harness*—the layer managing tools, permissions, and state—over its lifecycle [4]. These suites define *what* to detect; our benchmark defines *how* detection should be measured at the trajectory level, and imports their labeled traces as anomaly sources rather than re-inventing scenarios.

*Defenses and their audit blind spot.* Defenses cluster into three designs. Input-layer hardening restructures queries or optimizes model preferences to resist injection [6, 7]. Plan- and runtime-layer defenses constrain *what the agent may do*: IPIGuard checks tool calls against a tool-dependency graph derived from the plan [2], ToolSafe installs a proactive step-level guardrail with feedback [30], AgentDoG provides a diagnostic guardrail framework for agent safety [20], NEXUS enforces structured runtime safety for tool use [15], AgentGuard repurposes the orchestrator for safety evaluation of tool orchestration [5], and WebMCP-Phalanx enforces trust boundaries for browser-integrated agents [17]. All of these are step- or plan-scoped. Notably, even the most structural of them, IPIGuard, checks calls against the *intended* plan rather than learning what anomalous execution *looks like* across a corpus of real

traces; none learns a detector over observed trajectory structure. Our benchmark is precisely the substrate such learned structural detectors require, and our disclosure ablation quantifies how much signal each defense granularity leaves on the table.

*Benchmark validity and the closest work.* Two recent audits sharpen the stakes. Passing-the-Test shows prompt-injection detectors evaluated on the distributions they were tuned on lose most of their advantage out of distribution [25], and a validity audit of agent-safety benchmarks documents construct-validity failures—scores that reflect capability rather than safety [44]—echoing concerns raised for agent evaluation generally [29]. On the data side, AutoDojo generates fresh prompt-injection scenarios to escape static test sets [28], and PASTABench assesses agents over *sequential trajectories* [37]. The closest work to ours is the AgentDrift/DriftNet pair: AgentDrift publishes step-labeled injection-hijacked trajectories, and DriftNet is a dual-head trajectory transformer that detects and localizes hijacks from the sequence of steps [31, 32]. Our contribution is orthogonal and composable with theirs: we compile trajectories into heterogeneous provenance graphs, generalize the anomaly definition beyond injection to malicious use and over-privilege, and evaluate *graph-level* detectors under generalization, transfer, and mimicry controls that sequence-based evaluation does not exercise. To our knowledge, no prior benchmark treats the agent trajectory as a provenance graph or audits it with graph-level anomaly detection.

## 3 RELATED WORK: GRAPH ANOMALY DETECTION

*From node scores to learned structural detectors.* Graph anomaly detection (GAD) matured from classical outlier methods operating on graph-derived features [21, 50] into deep detectors with dedicated benchmarks: BOND standardized unsupervised outlier-node detection on attributed graphs [22], and GADBench did the same for supervised detection, showing simple supervised baselines are embarrassingly strong [38]. A structural thread then rethought what signal detectors should model: B-GAD links anomalousness to a homophily gap and resists heterophily [39]; GDN targets heterophily through graph spectrum alignment [11]; GAD-NR reconstructs neighborhoods rather than node features to expose anomalous connectivity [35]; ADA-GAD denoises the anomaly signal itself inside the autoencoder [14]; SmoothGNN smooths anomalies out of message passing to sharpen their contrast [10]; and dynamic-spectral detectors adapt filters to evolving structure [51]. These advances share an assumption that does not hold on agent traces: a large, mostly homogeneous graph where anomalies are rare nodes. Agent trajectory graphs are small, fully typed, temporally ordered, and anomalous *as wholes*, which is the regime of graph-level detection.

*Graph-level anomaly detection.* Graph-level GAD scores whole graphs against a distribution of normal graphs, surveyed comprehensively by [34]. The newest methods exploit language-aligned semantics and foundation-model transfer: GLASS aligns graph structure with language priors under a spherical scoring rule for transferable detection [43]; AnomalyGFM and OWLEYE target zero- and few-shot anomaly detection across domains [33, 52]; and AD-LLM benchmarks LLMs as anomaly detectors on tabular and graph tasks [47]. What is striking is the testbed monoculture: GLAD benchmarks are built from molecules, program graphs, and small citation clusters—graphs without execution semantics, without sensitive operations, and without an adversarial generative process producing the anomalies. Agent trajectories break all three assumptions: nodes are typed actions and artifacts, edges encode argument flow under an adversary, and the anomaly families are generated by replaying actual attacks. Our benchmark therefore gives GLAD a genuinely new testbed class, and conversely supplies the agent-safety community with detector families that exploit exactly the structure that step-level guardrails ignore.

*Adversarial pressure on graph detectors.* GAD methods are also attacked, and the attack literature maps one-to-one onto our robustness protocol. Node injection attacks insert malicious nodes whose attributes camouflage them into the normal class [16, 40]; multi-target variants coordinate injected subgraphs against fraud detectors [8]; and the newest work uses LLMs to generate text-level attributes for injected nodes that evade LLM-enhanced detectors [26, 27]. This is precisely the threat our mimicry protocol instantiates for trajectories: an adversary that splices benign-looking structure around the malicious payload. Prior GAD robustness work, however, injects nodes into *large* graphs to poison a detector; in our setting the adversary controls the entire trajectory-generation process, which is strictly stronger, and our protocol measures the resulting degradation explicitly rather than leaving it as future work.

## 4 RELATED WORK: PROVENANCE GRAPHS IN SYSTEMS SECURITY

*The provenance bet.* Systems security made the structural bet long before agents existed: whole-system provenance graphs record causal dependencies among processes, files, sockets, and syscalls, and whole-graph detectors reason over them to find stealthy intrusions invisible in any single event [53]. The paradigm supports streaming deployment [13] and efficient learned detectors over provenance subgraphs [45]. Crucially, the community also learned its limits the hard way: mimicry attacks that interleave benign activity around a small malicious backbone—preserving degrees, paths, and other structural statistics—defeat provenance-graph detectors, motivating detection over graph *semantics* and continuous evaluation under adaptive adversaries [12].

*Transferring the bet to the agentic Web.* Agent tool-call traces are, in a precise sense, application-layer provenance: an LLM process reads observations, transforms them into decisions, and emits tool invocations whose results become subsequent observations. But the transfer is not mechanical, and three differences define our design space. First, agent graphs are *typed semantically* rather than by syscall class: a node is a payment call or a file write, and its meaning binds to the task, so schema design is a first-class contribution rather than a fixed OS abstraction. Second, the unit of analysis is one user session—graphs of tens of nodes—rather than weeks of host activity, so detectors must operate on small graphs where whole-graph classification, not node scoring, is the natural inference. Third, the adversary does not merely inject structure; it generates the trajectory through a compromised controller, so anomaly families must

be produced by *replaying real attacks*, which is what our data engine does. IPIGuard is the closest bridge: it represents tool dependencies as a graph, but as a fixed defense-time plan constraint rather than a learned detector over observed provenance [2]. We supply the missing half—corpus-level, learned, whole-graph auditing—and we import the mimicry lesson directly into our evaluation protocol instead of rediscovering it after deployment.

# 5 TRAJECTORY PROVENANCE GRAPHS

## 5.1 Setting and Notation

We consider an LLM agent executing a task specified by an instruction $u$ over an environment $\mathcal{E}$ with tool catalog $\mathcal{T}$. A trajectory $\tau = (s_1, \ldots, s_K)$ is a sequence of $K$ steps, where step $s_k$ records the model turn, the tool call $(t_k, a_k, r_k)$ consisting of a tool $t_k \in \mathcal{T}$, arguments $a_k$, and result $r_k$, and the observation channel through which $r_k$ entered the context. A trajectory is *benign* when the sequence of calls serves $u$ within the privileges the task grants, and *anomalous* otherwise. The auditing task is to assign each trajectory a score $f(G) \in [0, 1]$ indicating anomaly, given its compiled graph $G$, and to localize the responsible steps with node-level scores.

We compile each trajectory into a heterogeneous attributed graph $G = (V, E, \phi, \psi)$ with node-type function $\phi : V \to \mathcal{N}$ and edge-type function $\psi : E \to \mathcal{R}$. Compilation is a deterministic function of the trace: no step of construction depends on the label, so benign and anomalous trajectories pass through the identical compiler. This discipline is what makes the benchmark auditable in turn—a compiler that looked at labels would smuggle the answer into the features.

## 5.2 Graph Schema

Table 1 defines the schema. The design follows one principle: every edge must encode a relation that either an injection hijack or an over-privileged action must ultimately violate. `flows` edges carry the dataflow of an argument datum into a later call, so the signature of an injection hijack—an untrusted `result` whose content steers a chain of calls ending at a sensitive `sink`—appears as a typed path $result \to step \to call \rightsquigarrow sink$. Over-privilege appears as a `call` whose `targets` edge points at a sink class that the task's grant does not cover, or whose authorizing subgraph (an upstream read or user confirmation) is absent. Malicious use, by contrast, may preserve entirely regular structure; we return to this asymmetry when defining the anomaly families, because it is what makes the benchmark discriminative rather than trivially solvable.

**Table 1: Node and edge types of the trajectory provenance graph.**

| Type | Kind | Semantics |
|---|---|---|
| `task` | node | The user instruction; root of the graph. |
| `step` | node | One model turn (reasoning or decision point). |
| `call` | node | One tool invocation instance. |
| `tool` | node | A catalog identity (e.g., `gmail.send`). |
| `entity` | node | An argument datum (file, account, URL, amount). |
| `result` | node | A tool observation returned to the agent. |
| `sink` | node | A sensitive-operation class (send, pay, delete, exfiltrate). |
| `invokes` | edge | $step \to call$: the model emits this call. |
| `uses` | edge | $call \to tool$: static tool identity. |
| `has_arg` | edge | $call \to entity$: argument binding. |
| `reads` | edge | $step \to result$: observation consumed. |
| `yields` | edge | $result \to call$: result feeds the next call. |
| `flows` | edge | $entity \to call$: dataflow of a datum into a call. |
| `then` | edge | $call \to call$: temporal order. |
| `targets` | edge | $call \to sink$: effect class of the call. |

Node attributes are typed per node class: `call` nodes carry the tool identity, argument schema, and replay timestamp; `entity` nodes carry a canonicalized value class (identifier, path, URL, currency) rather than the raw value, which bounds the feature space; `step` nodes carry text embeddings only in the structure-plus-text disclosure setting defined below. Edge attributes are minimal (temporal gap, argument slot index). The full graph for a typical trajectory contains [TBD—mean node/edge counts] nodes and [TBD] edges, so graph-level detectors face a small-graph regime with tens of nodes, not thousands.

## 5.3 Disclosure Control

Each compiled graph is released at two disclosure levels. In the *structure-only* level ($D_0$), all text is stripped: `step` nodes carry no content features, and `entity` nodes carry only value classes, mirroring what a defender can retain under strict logging policy. In the *structure-plus-text* level ($D_1$), `step` nodes additionally carry frozen sentence embeddings of the model turn, and `result` nodes carry embeddings of the observation. The ablation between $D_0$ and $D_1$ isolates the benchmark's central scientific question—how much auditing signal lives in structure alone—and simultaneously serves a deployment purpose, since many operators cannot or will not log raw prompts. All comparisons across detector families are run at both levels under identical splits.

## 5.4 Anomaly Families

The benchmark couples three anomaly families under one schema, chosen so that their signatures stress different parts of the graph.

*A1: injection-hijacked trajectories.* Following the step-labeled construction of AgentDrift, an A1 trajectory contains a hijack frontier: an observation in which an indirect injection payload is embedded, after which one or more calls service the attacker's goal rather than the user's task [31]. Benign counterparts are the unattacked replays of the same tasks [9]. A1 anomalies are structurally visible in $D_0$ *if and only if* the hijack altered the call pattern—which is exactly what our results measure, since a hijack that leaves the call sequence untouched is invisible to structure by construction.

*A2: malicious-use trajectories.* Drawn from AgentHarm's harmful task suite, A2 trajectories execute a request that is itself harmful, using the same benign-looking tool choreography as normal work [3]. Because the execution pattern is regular, A2 is the family where structure alone should fail and text should be necessary; including it prevents the benchmark from rewarding detectors that merely flag any unusual call sequence, and it mirrors the mixed reality of deployment where structural and semantic violations co-occur.

*A3: over-privileged tool use.* Constructed by our data engine, A3 trajectories contain calls that exceed the privilege the task grants—sending data the task only permitted to read, writing where only reading was authorized, or invoking a tool outside the task's declared scope—operationalizing the over-privilege taxonomy of [46]. Crucially, A3 contains no injected payload: the trajectory is structurally plausible, the instruction is benign, and the violation lives in the relation between the task's grant and the call's effect, encoded in the `targets` and missing-authorization structure of the graph.

### 5.5 Detection Formulation

Detectors receive a corpus $\{G_i\}$ at a fixed disclosure level and must return $f(G_i)$ for held-out graphs; family-aware supervised methods may additionally use training-set labels, but the LOAO protocol restricts which families are visible at training time (Section 6). Localization is optional but scored: detectors may emit node-level scores, which we evaluate against the step-level hijack labels of A1 [31]. This formulation deliberately mirrors graph-level GAD practice [34] while adding the adversarial evaluation axes that agent safety demands, and it admits every detector family we evaluate, from hand-crafted rules to graph foundation models [33, 52].

## 6 BENCHMARK PROTOCOL

### 6.1 Splits and Generalization Controls

The benchmark publishes a canonical split of benign and anomalous trajectories per environment, stratified so that no task template appears in both train and test. Beyond the canonical split, the protocol defines three controls that respond directly to documented failure modes of prior safety evaluation. First, *leave-one-attack-family-out* (LOAO): for each family $a \in \{A1, A2, A3\}$, detectors are trained on benign data plus the two other families and evaluated on $a$, measuring whether a detector tuned on known threats generalizes to unseen ones—the property that [25] shows prompt-injection detectors lose when evaluation distributions shift. Second, *cross-environment transfer*: detectors are trained on replayed environments and evaluated on a held-out environment whose tool catalog and task composition differ, with no re-tuning. Third, *contamination control*: the canonical split is constructed so that textual content of benign and anomalous trajectories is paraphrase-balanced, so that a text classifier cannot separate families by surface style alone; we verify this by reporting a text-only logistic-regression probe on $D_1$ inputs.

### 6.2 Mimicry Variants

Provenance-graph detectors in systems security are defeated by mimicry: adversaries that preserve benign structural statistics while executing the malicious payload [12]. Our protocol instantiates three mimicry variants over A1 and A3 trajectories. *M1 (structure splicing)* interleaves benign calls from matched benign trajectories around the malicious calls, preserving degree profiles and temporal regularity. *M2 (padding)* appends benign tail calls after the payload so that whole-graph statistics (counts, tool histograms) regress toward the benign distribution. *M3 (argument camouflage)* rewrites argument entities—splitting exfiltration across chunks, renaming accounts to benign-looking strings—so that entity-level features lose their signal while the dataflow graph is preserved. Because M3 changes entities but not edges, the comparison between M2 and M3 degradation cleanly attributes failure to surface features rather than structure. All variants are generated by deterministic scripts and released with the benchmark, so reported mimicry results are reproducible transformations, not researcher degrees of freedom.

### 6.3 Metrics

Detection is scored with AUROC and AUPRC (the latter primary, given class imbalance), plus Recall@$k$ at the operator-relevant operating points $k \in \{1\%, 5\%\}$ of the ranked audit queue and FPR@95TPR, which measures the false-alarm burden that a deployable guardrail imposes. Localization on A1 is scored as node-level AUC against step labels [31]. Generalization is summarized by the LOAO drop—the AUPRC gap between seen-family and unseen-family evaluation—and transfer is scored as the retained fraction of in-distribution AUPRC on the held-out environment. Audit economics reports GPU-seconds per trace and, where applicable, API cost per trace for LLM-judge baselines, because a detector that is accurate but costs more than the incident it prevents is not an auditing solution. All results are reported as mean ± standard deviation over five seeds; family-level comparisons use two-sided Wilcoxon signed-rank tests over seeds with Holm correction across the detector matrix.

### 6.4 Fairness Rules

Three rules keep the comparison meaningful. First, *identical inputs*: every detector consumes the same compiled graphs at the same disclosure level; no baseline may read the raw trace. Second, *tuning budget parity*: hyperparameter search is bounded identically across detectors, and the search space is published. Third, *no family peeking*: unsupervised methods never see family labels; supervised methods see them only under the LOAO restriction. The reference detector is held to the same rules as every baseline—its role is to be a strong, reproducible point on the detector map, not an advertised winner.

## 7 EXPERIMENTAL SETUP

### 7.1 Data

Table 2 summarizes the corpus. Benign and A1 trajectories are replayed in AgentDojo's workspace, travel, banking, and slack suites [9], with A1 hijack-step labels inherited from AgentDrift [31]; A2 trajectories are drawn from AgentHarm's harmful task suite with benign counterparts replayed for the same tool surfaces [3]; A3 trajectories are generated by our data engine over the same suites, parameterized by the over-privilege taxonomy of [46]. Replay uses locally hosted models—Qwen2.5-7B-Instruct and Qwen2.5-32B-Instruct-AWQ served by vLLM—so that every trajectory is regenerable on a single 24 GB consumer GPU, and the entire benchmark construction fits a [TBD] GPU-hour budget with no frontier-API dependency. The held-out transfer environment is [TBD—environment name], excluded from all training splits. Full per-suite counts, prompt templates, and grant definitions for A3 appear in the appendix.

**Table 2: Benchmark corpus. Counts are finalized on release; the split is stratified by task template.**

| Source | Benign | Anom. | Family | Disclosure |
|---|---|---|---|---|
| AgentDojo replays | [TBD] | — | benign | $D_0, D_1$ |
| AgentDrift hijacks | — | [TBD] | A1 | $D_0, D_1$ |
| AgentHarm suite | [TBD] | [TBD] | A2 | $D_0, D_1$ |
| Over-privilege engine | — | [TBD] | A3 | $D_0, D_1$ |
| Held-out environment | [TBD] | [TBD] | all | $D_0, D_1$ |

## 7.2 Detector Matrix

The matrix spans four families, all consuming identical graphs. *Rules* encode the auditor's prior directly: a call targeting a sensitive sink without an upstream authorizing read or confirmation flags the graph, and calls outside the task's declared grant likewise. *Shallow models over graph features* (IsolationForest, logistic regression from the PyOD toolbox [50]) consume hand-crafted whole-graph statistics: tool histograms, sink-touching counts, flow-path depths, and temporal regularity. *Sequence models* ignore the graph: an $n$-gram model over tool-call tokens, a dual-head trajectory transformer in the spirit of DriftNet [32], and an LLM judge that reads the textual trajectory and scores it, representing the per-trace version of current practice [19, 49]. *Graph neural detectors* include node-level GAD repurposed to whole-graph scores by pooling node anomalies (DOMINANT and CoLA from PyGOD [21]), zero-shot graph-level methods AnomalyGFM and OWLEYE [33, 52], the transferable graph-level detector GLASS [43], a supervised heterogeneous GNN with graph pooling as an upper bound, and our reference detector [TBD—detector name and one-line architecture from user's draft], trained with the family labels subject to LOAO restriction.

## 7.3 Implementation and Protocol

All detectors are tuned under the budget parity of Section 6.4 with five seeds; text embeddings for $D_1$ use [TBD—encoder]; replay uses temperature [TBD] with fixed seeds per trajectory. Experiments run on a single 24 GB consumer GPU; node-level and shallow detectors run on CPU. Statistical reporting follows Section 6.3: mean ± standard deviation over seeds, Wilcoxon signed-rank tests with Holm correction, and exact per-suite numbers in the appendix.

# 8 RESULTS

## 8.1 Main Detection Results

Table 3 reports canonical-split detection at both disclosure levels. The pattern to interrogate is not the leaderboard but three comparisons: rules versus learned shallow models (does the auditor's prior already suffice?); sequence models versus graph detectors at matched input (does structure add anything over order?); and $D_0$ versus $D_1$ (what does text buy?). [TBD—headline numbers: best $D_0$ AUPRC overall and per family; the $D_1$ increment; whether the supervised upper bound closes the gap to the reference detector.]

## 8.2 Generalization: LOAO and Transfer

Table 4 reports leave-one-attack-family-out generalization and cross-environment transfer. The diagnostic quantity is the LOAO drop: a detector whose advantage evaporates on its held-out family has memorized a threat rather than learned a structural property of compromise. The transfer column asks the deployment question—does a detector trained on replayed environments retain its advantage when the tool catalog changes? [TBD—LOAO drop per detector; which family is hardest held-out; transfer retention fraction.]

**Table 3: Canonical-split detection (AUPRC / AUROC, mean ± std over five seeds).**

| Detector | $D_0$ (structure-only) | | $D_1$ (structure+text) | |
|---|---|---|---|---|
| | AUPRC | AUROC | AUPRC | AUROC |
| Rules | [TBD] | [TBD] | — | — |
| Graph features + iForest | [TBD] | [TBD] | — | — |
| $n$-gram | [TBD] | [TBD] | [TBD] | [TBD] |
| Trajectory transformer | [TBD] | [TBD] | [TBD] | [TBD] |
| LLM judge | — | — | [TBD] | [TBD] |
| DOMINANT (pooled) | [TBD] | [TBD] | [TBD] | [TBD] |
| CoLA (pooled) | [TBD] | [TBD] | [TBD] | [TBD] |
| AnomalyGFM (zero-shot) | [TBD] | [TBD] | [TBD] | [TBD] |
| OWLEYE (zero-shot) | [TBD] | [TBD] | [TBD] | [TBD] |
| GLASS | [TBD] | [TBD] | [TBD] | [TBD] |
| HetGNN upper bound | [TBD] | [TBD] | [TBD] | [TBD] |
| Reference detector | [TBD] | [TBD] | [TBD] | [TBD] |

**Table 4: LOAO generalization and cross-environment transfer (AUPRC).**

| Detector | hold A1 | hold A2 | hold A3 | mean drop | transfer |
|---|---|---|---|---|---|
| Rules | [TBD] | [TBD] | [TBD] | [TBD] | [TBD] |
| Graph features + iForest | [TBD] | [TBD] | [TBD] | [TBD] | [TBD] |
| Trajectory transformer | [TBD] | [TBD] | [TBD] | [TBD] | [TBD] |
| GLASS | [TBD] | [TBD] | [TBD] | [TBD] | [TBD] |
| HetGNN upper bound | [TBD] | [TBD] | [TBD] | [TBD] | [TBD] |
| Reference detector | [TBD] | [TBD] | [TBD] | [TBD] | [TBD] |

## 8.3 Mimicry Robustness

Table 5 reports degradation under the three mimicry variants of Section 6.2. Because M2 degrades surface statistics while M3 preserves them, the M2–M3 contrast attributes failures to surface features versus structure. If structural detectors degrade as sharply as shallow ones, the provenance bet fails on agent trajectories in its naive form, and the field's assumption that structure is the robust signal needs revision—either outcome is a finding the benchmark is designed to expose. [TBD—degradation per variant; the M2–M3 contrast; whether any detector retains advantage under M1 splicing.]

## 8.4 Localization and Audit Economics

Localization turns a detector into a workflow: an auditor triages flagged traces, and step-level scores direct the first read. We score node-level AUC against AgentDrift's step labels on A1 [31]. Audit economics, per Section 6.3, reports GPU-seconds per trace and API cost where applicable; the LLM judge is expected to dominate cost by orders of magnitude, which matters because the audit queue in

**Table 5: Mimicry robustness on A1 and A3 (AUPRC, relative to canonical split).**

| Detector | M1 splicing | M2 padding | M3 camouflage |
|---|---|---|---|
| Graph features + iForest | [TBD] | [TBD] | [TBD] |
| Trajectory transformer | [TBD] | [TBD] | [TBD] |
| GLASS | [TBD] | [TBD] | [TBD] |
| Reference detector | [TBD] | [TBD] | [TBD] |

deployment is long and mostly benign. [TBD—localization AUC; cost table; cost–accuracy frontier statement.]

### 8.5 Analysis

Three analyses complete the map. First, *family structure*: we visualize the compiled graphs of top-ranked false positives and false negatives, reporting which schema relations drive errors—our working hypothesis, to be confirmed or refuted by the [TBD] experiments, is that `flows` paths into `sink` nodes carry the A1 signal while A3 lives in missing-authorization structure. Second, *disclosure policy*: the $D_0$–$D_1$ gap quantifies how much audit quality an operator forfeits by declining to log prompt text, an operational trade-off no prior agent-safety benchmark measures. Third, *sensitivity*: we vary benign-trajectory contamination, family mixing ratios, and embedding choices, reporting stability of the detector ranking under perturbation so that the benchmark's conclusions do not hinge on one configuration.

## 9 CONCLUSION

We introduced TraceGraph-Bench, the first benchmark that audits LLM agent tool-call trajectories as heterogeneous provenance graphs with graph-level anomaly detection. The benchmark couples three anomaly families—injection hijack, malicious use, and over-privilege—under one typed graph schema, and its protocol builds the controls that recent audits of safety evaluation demand into the spine of the design: leave-one-attack-family-out generalization, cross-environment transfer, mimicry robustness in the lineage of provenance-graph attacks, a disclosure ablation separating structure from text, and audit-economics accounting. A systematized detector matrix, from rules to graph foundation models to a supervised heterogeneous-graph reference detector, provides the first map of this new problem, showing where structure alone suffices, where text is necessary, and where adaptive splicing breaks the structural bet. We release the compiler, traces, graphs, and harness to make trajectory-level auditing reproducible on consumer hardware.

## LIMITATIONS

Three limitations bound our claims. First, replayed trajectories reflect the model families and suites we could host locally; behaviors of frontier agents may differ, which is why the benchmark publishes the compiler as a protocol rather than a fixed dataset. Second, our mimicry variants are scripted transformations rather than a learned adversary; they bound robustness below a fully adaptive attacker, and we deliberately omit a feedback-loop attack arm from this version. Third, A3 over-privilege is defined against explicit grants; real deployments often lack such specifications, and extending the family to inferred grants is future work.

## ETHICS

This work studies defensive auditing of agent behavior and creates no new attack capability beyond what is already public in the source benchmarks we replay; injection payloads and harmful-task specifications are inherited from AgentDojo, AgentDrift, and AgentHarm under their respective licenses and are used solely to construct labeled audit data. All trajectories are synthetic or replayed in sandboxed environments with simulated tools; no real user data, accounts, or systems are involved. We will release data and code under licenses that permit defensive research while preserving the upstream benchmark licenses.

## REFERENCES


[1] Faruk Alpay and Taylan Alpay. 2026. AgentSecBench: Measuring Prompt Injection, Privacy Leakage, and Tool-Use Integrity in LLM Agents. In *arXiv (Cornell University)*. Cornell University. https://doi.org/10.48550/arxiv.2605.26269

[2] Hongxin An, Jinghuai Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. 2025. IPIGuard: A Novel Tool Dependency Graph-Based Defense Against Indirect Prompt Injection in LLM Agents. *Conference on Empirical Methods in Natural Language Processing (EMNLP)* (2025), 1023–1039. https://doi.org/10.18653/v1/2025.emnlp-main.53

[3] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin L. Wang, Dan Hendrycks, Andy Zou, J. Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. 2024. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. In *arXiv (Cornell University)*. Cornell University. https://doi.org/10.48550/arxiv.2410.09024

[4] Yajing Bai, Jinhao Duan, Jie Peng, Xianfeng Wu, Sijia Liu, Song Wang, and Tianlong Chen. 2026. HarnessRisk: A Lifecycle-Oriented Benchmark for Agent Harness Safety. In *arXiv (Cornell University)*. Cornell University. https://doi.org/10.48550/arxiv.2608.17597

[5] Jizhou Chen and Samuel Lee Cong. 2025. AgentGuard: Repurposing Agentic Orchestrator for Safety Evaluation of Tool Orchestration. arXiv:2502.09809. https://doi.org/10.48550/arxiv.2502.09809 arXiv:2502.09809

[6] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2024. StruQ: Defending Against Prompt Injection with Structured Queries. arXiv:2402.06363. https://doi.org/10.48550/arxiv.2402.06363 arXiv:2402.06363

[7] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2024. SecAlign: Defending Against Prompt Injection with Preference Optimization. arXiv:2410.05451. https://doi.org/10.48550/arxiv.2410.05451 arXiv:2410.05451

[8] Jinhyeok Choi, Heehyeon Kim, and Joyce Jiyoung Whang. 2024. Unveiling the Threat of Fraud Gangs to Graph Neural Networks: Multi-Target Graph Injection Attacks Against GNN-Based Fraud Detectors. arXiv:2412.18370. https://doi.org/10.48550/arxiv.2412.18370 arXiv:2412.18370

[9] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *neural information processing systems* (2024), 82895–82920. https://doi.org/10.52202/079017-2636

[10] Xiangyu Dong, Xingyi Zhang, Yanni Sun, Lei Chen, Mingxuan Yuan, and Sibo Wang. 2025. SmoothGNN: Smoothing-aware GNN for Unsupervised Node Anomaly Detection. In *Proceedings of the ACM on Web Conference 2025*. ACM, 1225–1236. https://doi.org/10.1145/3696410.3714615

[11] Yuan Gao, Xiang Wang, Xiangnan He, Zhenguang Liu, Huamin Feng, and Yongdong Zhang. 2023. Addressing Heterophily in Graph Anomaly Detection: A Perspective of Graph Spectrum. In *Proceedings of the ACM Web Conference 2023*. ACM, 1528–1538. https://doi.org/10.1145/3543507.3583268

[12] Akul Goyal, Xueyuan Han, Gang Wang, and Adam Bates. 2023. Sometimes, You Aren't What You Do: Mimicry Attacks against Provenance Graph Host Intrusion Detection Systems. In *Proceedings 2023 Network and Distributed System Security Symposium*. Internet Society. https://doi.org/10.14722/ndss.2023.24207

[13] Akul Goyal, Jason Liu, Adam Bates, and Gang Wang. 2024. ORCHID: Streaming Threat Detection over Versioned Provenance Graphs. arXiv:2408.13347. https://doi.org/10.48550/arxiv.2408.13347 arXiv:2408.13347

[14] Junwei He, Qianqian Xu, Yangbangyan Jiang, Zitai Wang, and Qingming Huang. 2024. ADA-GAD: Anomaly-Denoised Autoencoders for Graph Anomaly Detection. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 8 (2024), 8481–8489. https://doi.org/10.1609/aaai.v38i8.28691

[15] Elias Hossain, Md Mehedi Hasan Nipu, Tasfia Nuzhat Ornee, Rajib Rana, and Niloofar Yousefi. 2026. NEXUS: Structured Runtime Safety for Tool-Using LLM Agents. arXiv:2607.19356. https://doi.org/10.48550/arxiv.2607.19356 arXiv:2607.19356

[16] Mingxuan Ju, Yujie Fan, Yanfang Ye, and Liang Zhao. 2022. Black-box Node Injection Attack for Graph Neural Networks. arXiv:2202.09389. https://doi.org/10.48550/arxiv.2202.09389 arXiv:2202.09389

[17] Lin-Fa Lee, YI-YU Chang, and Kuo-Hui Yeh. 2026. WebMCP-Phalanx: Enforcing and Characterizing Trust Boundaries for Browser-Integrated LLM Agents. arXiv:2608.24017. https://doi.org/10.48550/arxiv.2608.24017 arXiv:2608.24017

[18] Yunseok Lee, Yunji Kim, and Woojin Lee. 2026. Will the User Ever Know? Covert Indirect Prompt Injection Attacks on Tool-Using LLM Agents. arXiv:2608.30362. https://doi.org/10.48550/arxiv.2608.30362 arXiv:2608.30362

[19] Xu Li, Simon Yu, Minzhou Pan, Yiyou Sun, Bo Li, Dawn Song, Xue Lin, and Weiyan Shi. 2026. Unsafer in Many Turns: Benchmarking and Defending Multi-Turn Safety Risks in Tool-Using Agents. In *arXiv (Cornell University)*. Cornell University. https://doi.org/10.48550/arxiv.2602.13379

[20] Dongrui Liu, Qihan Ren, Chen Qian, Shuai Shao, Yuejin Xie, Yu Li, Zhonghao Yang, Haoyu Luo, Peng Wang, Qingyu Liu, Binxin Hu, Ling Tang, Jilin Mei, Dadi Guo, Leitao Yuan, Junyao Yang, Guanxu Chen, Qihao Lin, Yi Yu, Bo Zhang, Jiaxuan Guo, Jie Zhang, Wenqi Shao, Huiqi Deng, Zhiheng Xi, Wenjie Wang, Wenxuan Wang, Wen Shen, Zhikai Chen, Haoyu Xie, Jialing Tao, Juntao Dai, Jiaming Ji, Zhongjie Ba, Linfeng Zhang, Yong Liu, Quanshi Zhang, Lei Zhu, Zhihua Wei, Hui Xue, Chaochao Lu, Jing Shao, and Xia Hu. 2026. AgentDoG: A Diagnostic Guardrail Framework for AI Agent Safety and Security. arXiv:2601.18491. https://doi.org/10.48550/arxiv.2601.18491 arXiv:2601.18491

[21] Kay Liu, Yingtong Dou, Xueying Ding, Xiyang Hu, Ruitong Zhang, Hao Peng, Lichao Sun, and Philip S. Yu. 2022. PyGOD: A Python Library for Graph Outlier Detection. arXiv:2204.12095. https://doi.org/10.48550/arxiv.2204.12095 arXiv:2204.12095

[22] Kay Liu, Yingtong Dou, Yue Zhao, Xueying Ding, Xiyang Hu, Ruitong Zhang, Kaize Ding, Canyu Chen, Hao Peng, Kai Shu, Lichao Sun, Jundong Li, George H. Chen, Zhihao Jia, and Philip S. Yu. 2022. BOND: Benchmarking Unsupervised Outlier Node Detection on Static Attributed Graphs. arXiv:2206.10071. https://doi.org/10.48550/arxiv.2206.10071 arXiv:2206.10071

[23] Xiaogeng Liu, Zhiyuan Yu, Yizhe Zhang, Ning Zhang, and Chaowei Xiao. 2024. Automatic and Universal Prompt Injection Attacks against Large Language Models. arXiv:2403.04957. https://doi.org/10.48550/arxiv.2403.04957 arXiv:2403.04957

[24] Yinuo Liu, Xilong Wang, Ruohan Xu, Yuqi Jia, and Neil Zhenqiang Gong. 2025. WAInjectBench: Benchmarking Prompt Injection Detections for Web Agents. arXiv:2510.01354. https://doi.org/10.48550/arxiv.2510.01354 arXiv:2510.01354

[25] Zhuowen Liu. 2026. Passing the Test You Trained On: Re-evaluating Prompt-Injection Detectors for LLM Agents. arXiv:2610.03448. https://doi.org/10.48550/arxiv.2610.03448 arXiv:2610.03448

[26] Yuefei Lyu, Chaozhuo Li, Xi Zhang, and Tianle Zhang. 2025. Navigating the Black Box: Leveraging LLMs for Effective Text-Level Graph Injection Attacks. arXiv:2506.13276. https://doi.org/10.48550/arxiv.2506.13276 arXiv:2506.13276

[27] Jiaji Ma, Puja Trivedi, and Danai Koutra. 2025. GRAPHTEXTACK: A Realistic Black-Box Node Injection Attack on LLM-Enhanced GNNs. arXiv:2511.12423. https://doi.org/10.48550/arxiv.2511.12423 arXiv:2511.12423

[28] Xinhang Ma, Taoran Li, Chaowei Xiao, Zhiyuan Yu, Ning Zhang, and Yevgeniy Vorobeychik. 2026. AutoDojo: A Generative Benchmark for Evaluating Prompt Injection Defenses in LLM Agents. arXiv:2606.15057. https://doi.org/10.48550/arxiv.2606.15057 arXiv:2606.15057

[29] Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. 2025. Evaluation and Benchmarking of LLM Agents: A Survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*. ACM, 6129–6139. https://doi.org/10.1145/3711896.3736570

[30] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. 2026. ToolSafe: Enhancing Tool Invocation Safety of LLM-based agents via Proactive Step-level Guardrail and Feedback. arXiv:2601.10156. https://doi.org/10.48550/arxiv.2601.10156 arXiv:2601.10156

[31] Asif Pinjari and Mithun Paul Saint-Germain. 2026. AgentDrift: A Step-Labeled Benchmark of Injection-Hijacked LLM Agent Trajectories. In *arXiv (Cornell University)*. Cornell University. https://doi.org/10.48550/arxiv.2609.06972

[32] Asif Pinjari and Mithun Paul Saint-Germain. 2026. DriftNet: A Dual-Head Trajectory Transformer for Detecting and Localizing Prompt Injection in LLM Agents. In *arXiv (Cornell University)*. Cornell University. https://doi.org/10.48550/arxiv.2609.10892

[33] Hezhe Qiao, Chaoxi Niu, Ling Chen, and Guansong Pang. 2025. AnomalyGFM: Graph Foundation Model for Zero/Few-shot Anomaly Detection. arXiv:2502.09254. https://doi.org/10.48550/arxiv.2502.09254 arXiv:2502.09254

[34] Hezhe Qiao, Hanghang Tong, Bo An, Irwin King, Charu Aggarwal, and Guansong Pang. 2024. Deep Graph Anomaly Detection: A Survey and New Perspectives. arXiv:2409.09957. https://doi.org/10.48550/arxiv.2409.09957 arXiv:2409.09957

[35] Amit Roy, Juan Shu, Jia Li, Carl Yang, Olivier Elshocht, Jeroen Smeets, and Pan Li. 2024. GAD-NR: Graph Anomaly Detection via Neighborhood Reconstruction. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. ACM, 576–585. https://doi.org/10.1145/3616855.3635767

[36] Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. 2025. Prompt Injection Attack to Tool Selection in LLM Agents. arXiv:2504.19793. https://doi.org/10.48550/arxiv.2504.19793 arXiv:2504.19793

[37] Jiapeng Sun, Yujin Zhou, Han Zhu, Pengcheng Wen, Jiayi Zhou, Sirui Han, and Yike Guo. 2026. PASTABench: Proactive Assessment of Sequential Trajectories for Agent Safety. arXiv:2609.28197. https://doi.org/10.48550/arxiv.2609.28197 arXiv:2609.28197

[38] Jianheng Tang, Fengrui Hua, Ziqi Gao, Peilin Zhao, and Jia Li. 2023. GADBench: Revisiting and Benchmarking Supervised Graph Anomaly Detection. arXiv:2306.12251. https://doi.org/10.48550/arxiv.2306.12251 arXiv:2306.12251

[39] Jianheng Tang, Jiajin Li, Ziqi Gao, and Jia Li. 2022. Rethinking Graph Neural Networks for Anomaly Detection. arXiv:2205.15508. https://doi.org/10.48550/arxiv.2205.15508 arXiv:2205.15508

[40] Shuchang Tao, Qi Cao, Huawei Shen, Yunfan Wu, Liang Hou, Fei Sun, and Xueqi Cheng. 2022. Adversarial Camouflage for Node Injection Attack on Graphs. arXiv:2208.01819. https://doi.org/10.48550/arxiv.2208.01819 arXiv:2208.01819

[41] Peiran Wang, Xinfeng Li, Chong Xiang, Jinghuai Zhang, Ying Li, Lixia Zhang, Xiaofeng Wang, and Yuan Tian. 2026. The Landscape of Prompt Injection Threats in LLM Agents: From Taxonomy to Analysis. arXiv:2602.10453. https://doi.org/10.48550/arxiv.2602.10453 arXiv:2602.10453

[42] Xilong Wang, John Bloch, Zedian Shao, Yuepeng Hu, Shuyan Zhou, and Neil Zhenqiang Gong. 2025. WebInject: Prompt Injection Attack to Web Agents. The 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025). arXiv:2505.11717 http://arxiv.org/abs/2505.11717v4

[43] Xudong Wang, Chris Ding, Tongxin Li, and Jicong Fan. 2026. GLASS: Graph-Language Alignment with Spherical Scoring for Transferable Graph-Level Anomaly Detection. arXiv:2609.05253. https://doi.org/10.48550/arxiv.2609.05253 arXiv:2609.05253

[44] Youting Wang, Xiao Han, Dingyan Shang, Yuan Tang, and Bowen Liu. 2026. Safety, or Just Capability? A Validity Audit of Agent-Safety Benchmarks. arXiv:2607.28685. https://doi.org/10.48550/arxiv.2607.28685 arXiv:2607.28685

[45] Boyuan Xu, Yiru Gong, Xiaoyu Geng, Yun Li, Cong Dong, Song Liu, Yuling Liu, Bo Jiang, and Zhigang Lu. 2024. ProcSAGE: an efficient host threat detection method based on graph representation learning. *Cybersecurity* 7, 1 (2024). https://doi.org/10.1186/s42400-024-00240-w

[46] Kaiyue Yang, Yuyan Bu, Jingwei Yi, Yuchi Wang, Biyu Zhou, Juntao Dai, Songlin Hu, and Yaodong Yang. 2026. When Lower Privileges Suffice: Investigating Over-Privileged Tool Selection in LLM Agents. arXiv:2606.20023. https://doi.org/10.48550/arxiv.2606.20023 arXiv:2606.20023

[47] Tiankai Yang, Yi Nian, Shawn Li, Ruiyao Xu, Yuangang Li, Jiaqi Li, Zhuo Xiao, Xiyang Hu, Ryan Rossi, Kaize Ding, Xia Hu, and Yue Zhao. 2024. AD-LLM: Benchmarking Large Language Models for Anomaly Detection. arXiv:2412.11142. https://doi.org/10.48550/arxiv.2412.11142 arXiv:2412.11142

[48] Junjie Ye, Sixian Li, Guanyu Li, Caishuang Huang, Songyang Gao, Yilong Wu, Qi Zhang, Tao Gui, and Xuanjing Huang. 2024. ToolSword: Unveiling Safety Issues of Large Language Models in Tool Learning Across Three Stages. *Annual Meeting of the Association for Computational Linguistics (ACL)* (2024), 2181–2211. https://doi.org/10.18653/v1/2024.acl-long.119

[49] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, XU Li-zhen, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. 2024. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. (2024), 1467–1490. https://doi.org/10.18653/v1/2024.findings-emnlp.79

[50] Yue Zhao, Zain Nasrullah, and Zheng Li. 2019. PyOD: A Python Toolbox for Scalable Outlier Detection. arXiv:1901.01588. https://doi.org/10.48550/arxiv.1901.01588 arXiv:1901.01588

[51] Jianbo Zheng, Chao Yang, Tairui Zhang, Longbing Cao, Bin Jiang, Xuhui Fan, Xiao ming Wu, and Xianxun Zhu. 2025. Dynamic Spectral Graph Anomaly Detection. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 12 (2025), 13410–13418. https://doi.org/10.1609/aaai.v39i12.33464

[52] Lecheng Zheng, Dongqi Fu, Zihao Li, and Jingrui He. 2026. OWLEYE: Zero-Shot Learner for Cross-Domain Graph Data Anomaly Detection. arXiv:2601.19102. https://doi.org/10.48550/arxiv.2601.19102 arXiv:2601.19102

[53] Michael Zipperle, Florian Gottwalt, Elizabeth Chang, and Tharam Dillon. 2022. Provenance-based Intrusion Detection Systems: A Survey. *Comput. Surveys* 55, 7 (2022), 1–36. https://doi.org/10.1145/3539605

# A REPRODUCIBILITY DETAILS

*Replay harness.* Trajectories are regenerated with vLLM serving Qwen2.5-7B-Instruct (bf16) and Qwen2.5-32B-Instruct-AWQ (4-bit) on a single 24 GB GPU; sampling uses temperature [TBD], top-$p$ [TBD], and a per-trajectory seed derived from the task identifier, so every published graph is regenerable bit-for-bit from the released scripts. AgentDojo suites are replayed with their original

task definitions and attack configurations [9]; A1 step labels are aligned to our `step` nodes from the AgentDrift release [31]; A2 trajectories replay AgentHarm's harmful tasks with their benign counterparts [3]; A3 generation scripts, grant definitions, and the over-privilege parameter grid derived from [46] are released with the compiler.

*Graph compiler.* Compilation is a deterministic three-pass pass over the trace JSONL: (1) node emission with canonicalized entity classes (identifier, path, URL, currency, free-text); (2) edge emission from the replay event log (invocation order, argument bindings, observation consumption); (3) attribute assembly under the requested disclosure level $D_0$ or $D_1$. Entity canonicalization rules and the full attribute dictionary are listed in [TBD—appendix table or repository pointer]. Mean graph statistics: [TBD] nodes, [TBD] edges, [TBD] s per trajectory on one CPU core.

*Hyperparameters.* Per-detector search spaces, budgets, and selected configurations are reported in [TBD—hyperparameter table], including the reference detector's training schedule, loss, and hardware footprint; five-seed runs use seeds {0–4} with shared data folds.

## B LICENSES AND ATTRIBUTION

The benchmark builds on AgentDojo (original license), AgentDrift, and AgentHarm releases; we redistribute only compiled graphs and regenerable scripts, not upstream assets, and each artifact carries its upstream license pointer. The over-privilege family is synthetic. Released code is under [TBD—license]; data under [TBD—license].